

Sorting

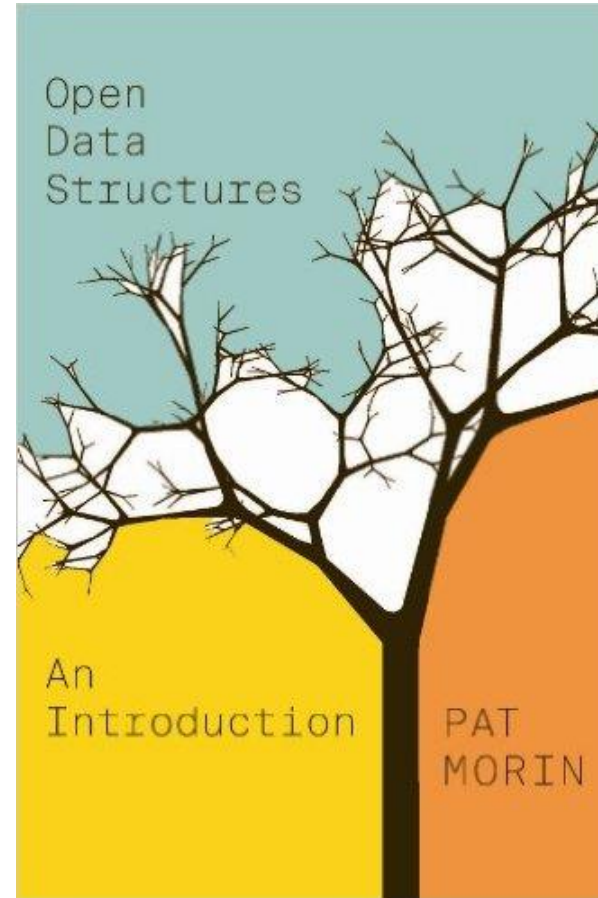
COMP 2402

Abstract Data Types & Algorithms

Readings

Today's class

- Counting and Radix Sort
 - Chapter 11.2



Comparison-Based Sorting

	Comparisons		In-place	Stable
Mergesort	$n \log(n)$	Worst-case	No	Yes
Quicksort	$1.38 n \log(n) + O(n)$ $O(n^2)$	Expected Worst-case	Yes	No
Heapsort	$2n \log(n) + O(n)$	Worst-case	Yes	No
Timsort	$O(n \log n)$ $O(n)$	Worst-case Best-case	No	Yes
Insertion Sort	$O(n^2)$ $O(n)$	Worst-case Best-case	Yes	Yes
Selection	$O(n^2)$	Worst-case	Yes	Yes

Lower Bound for Sorting

Theorem 11.5: For any deterministic comparison-based sorting algorithm A and any integer $n \geq 1$, there exists an input array a of length n such that $A(a)$ performs at least

$$\log(n!) = n \log(n) - O(n)$$

comparisons when sorting a .

Counting Sort

Ok, we now know that sorting is $\Omega(n \log(n))$.

So let's beat it!

Counting Sort

Suppose we have an array of size n that contains only integers in the range $[0, n)$.

Counting Sort

Suppose we have an array of size n that contains only integers in the range $[0, n)$.

How can we sort these numbers in $O(n)$ time?

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
	0	0	0	0	0	0	0	0	0	0	0
C	0	1	2	3	4	5	6	7	8	9	10

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j	↑										
C	0	0	0	0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[0] = 2 \rightarrow \text{increment } C[2]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	0	1	0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[0] = 2 \rightarrow \text{increment } C[2]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j		↑									
C	0	0	1	0	0	1	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[1] = 5 \rightarrow \text{increment } C[5]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	1	0	0	1	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[2] = 1 \rightarrow \text{increment } C[1]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	1	1	0	1	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[3] = 3 \rightarrow \text{increment } C[3]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	2	1	0	1	0	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[4] = 2 \rightarrow \text{increment } C[2]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	2	1	0	1	1	0	0	0	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[5] = 6 \rightarrow \text{increment } C[6]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j							↑				
C	0	1	2	1	0	1	1	0	0	1	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[6] = 9 \rightarrow \text{increment } C[9]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	2	1	0	1	1	1	0	1	0
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[7] = 7 \rightarrow \text{increment } C[7]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j									↑		
C	0	1	2	1	0	1	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[8] = 10 \rightarrow \text{increment } C[10]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j										↑	
C	0	1	3	1	0	1	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[9] = 2 \rightarrow \text{increment } C[2]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10

$A[j] = A[10] = 5 \rightarrow \text{increment } C[5]$

Counting Sort

Walk through A and increment $C[A[j]]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10

Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'											

Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0										



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1									



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4								



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5							



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5						



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7					



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8				



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8	9			



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8	9	9		



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8	9	9	10	



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8	9	9	10	11



Counting Sort

Walk through C and compute $C[j] += C[j - 1]$

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8	9	9	10	11

Counting Sort

Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8	9	9	10	11
B											

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7	8	9	9	10	11
B											

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

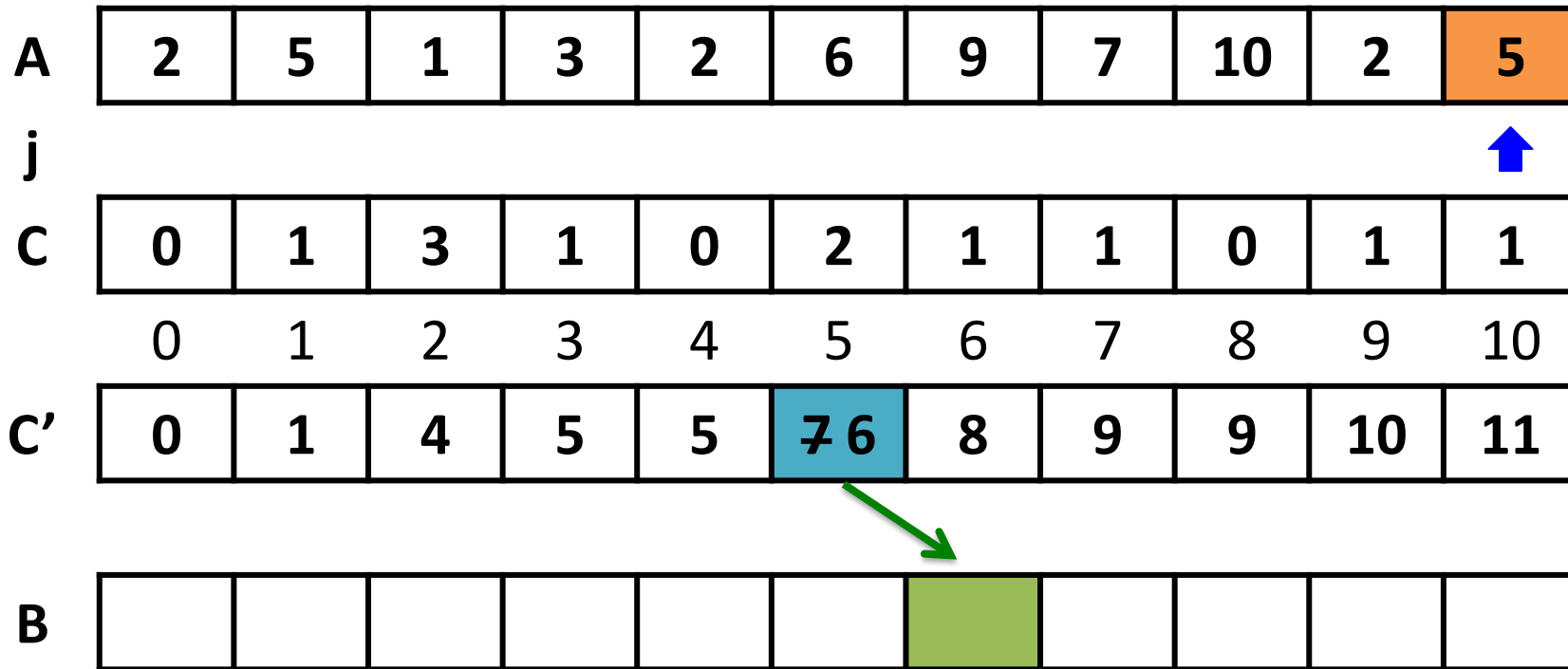
Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7 6	8	9	9	10	11
B											

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j											↑
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	7 6	8	9	9	10	11
B							5				

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

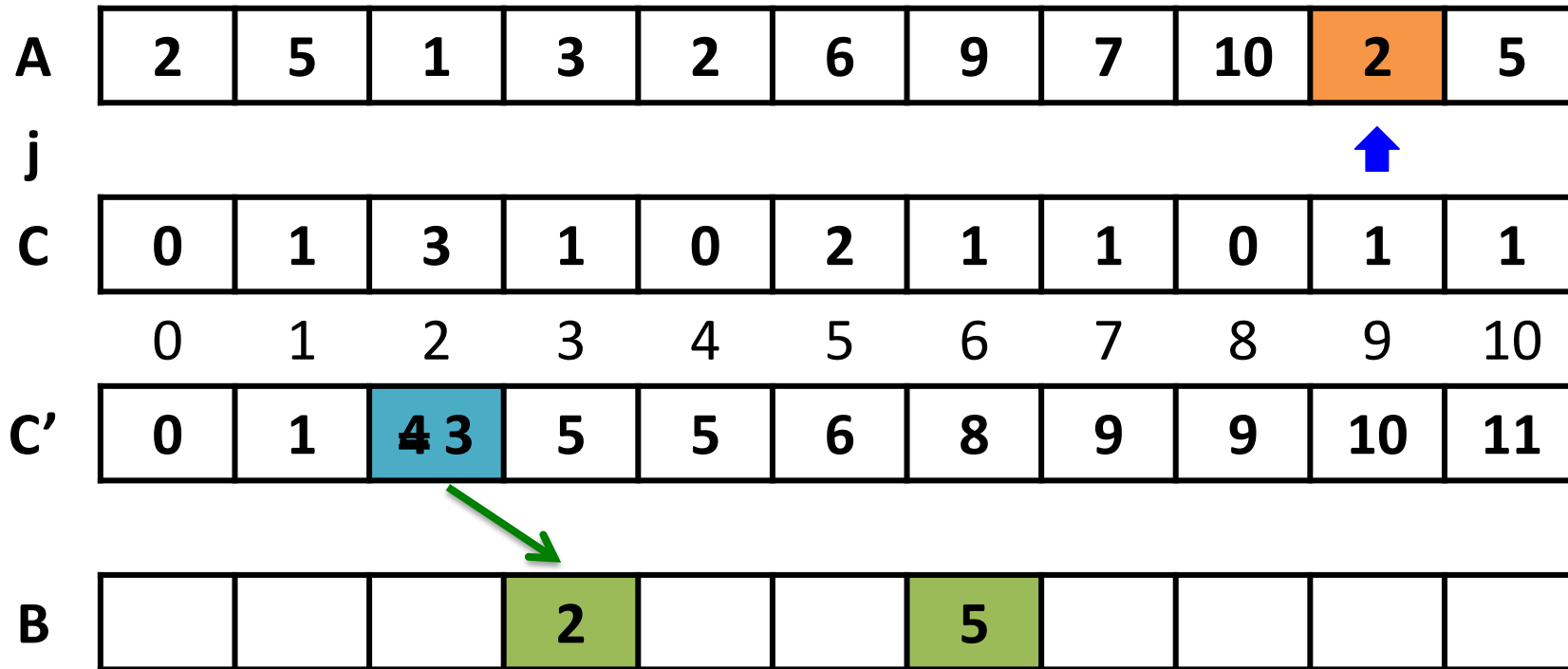
Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	4	5	5	6	8	9	9	10	11
B							5				

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

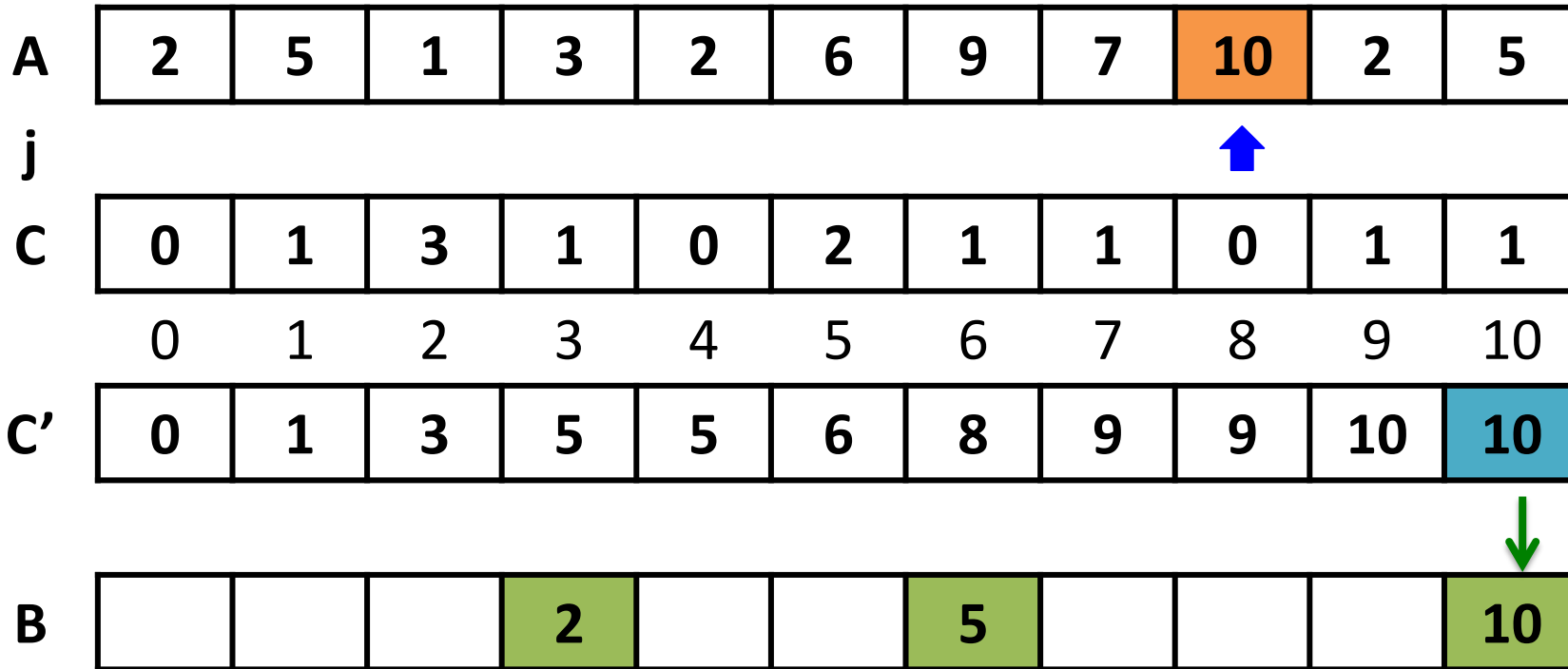
Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

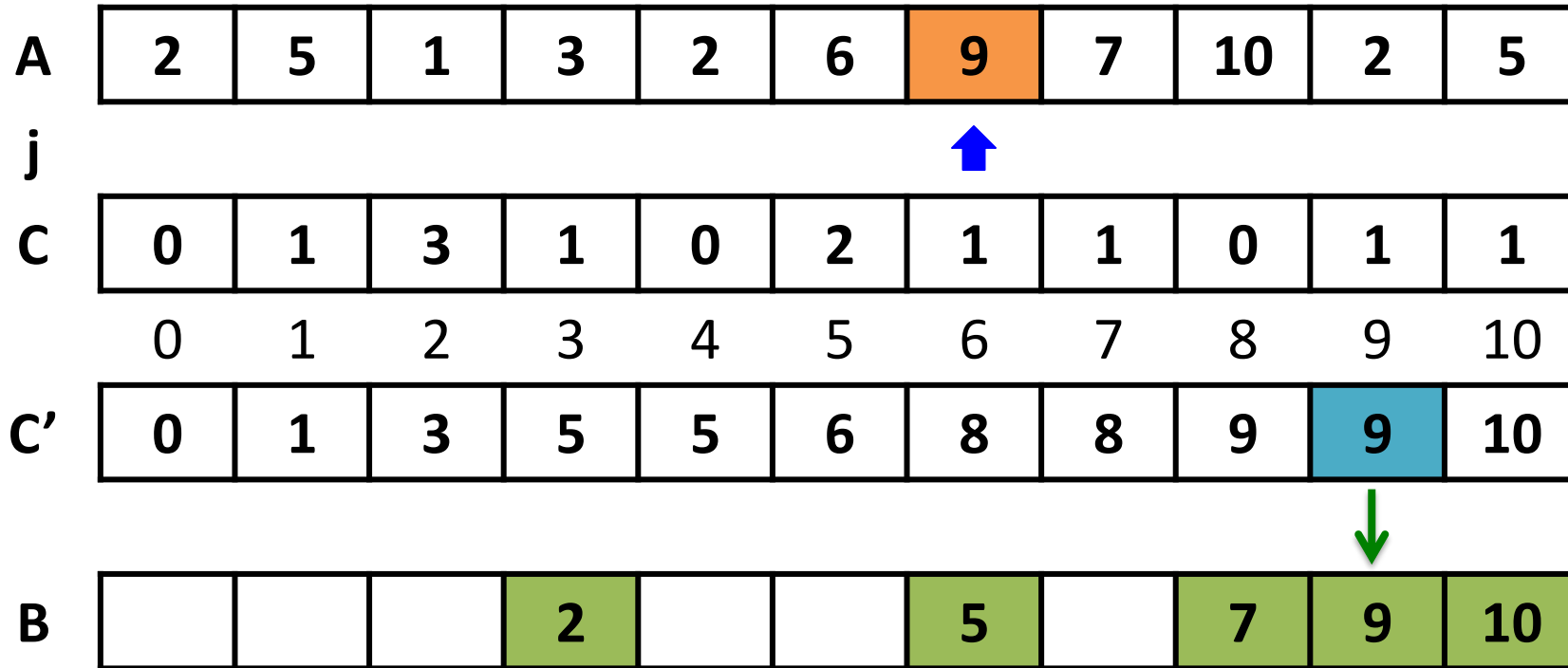
Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j								↑			
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	1	3	5	5	6	8	8	9	10	10
								↘			
B				2			5		7		10

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

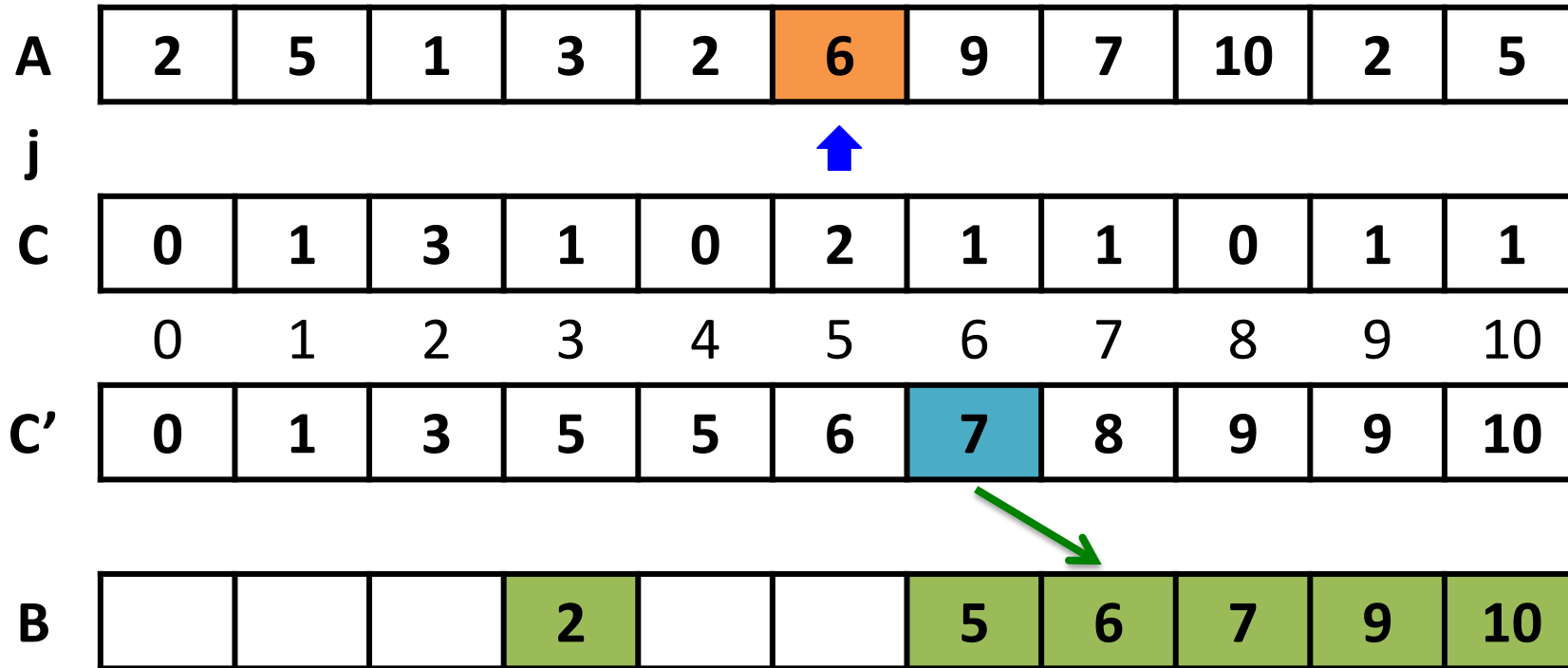
Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

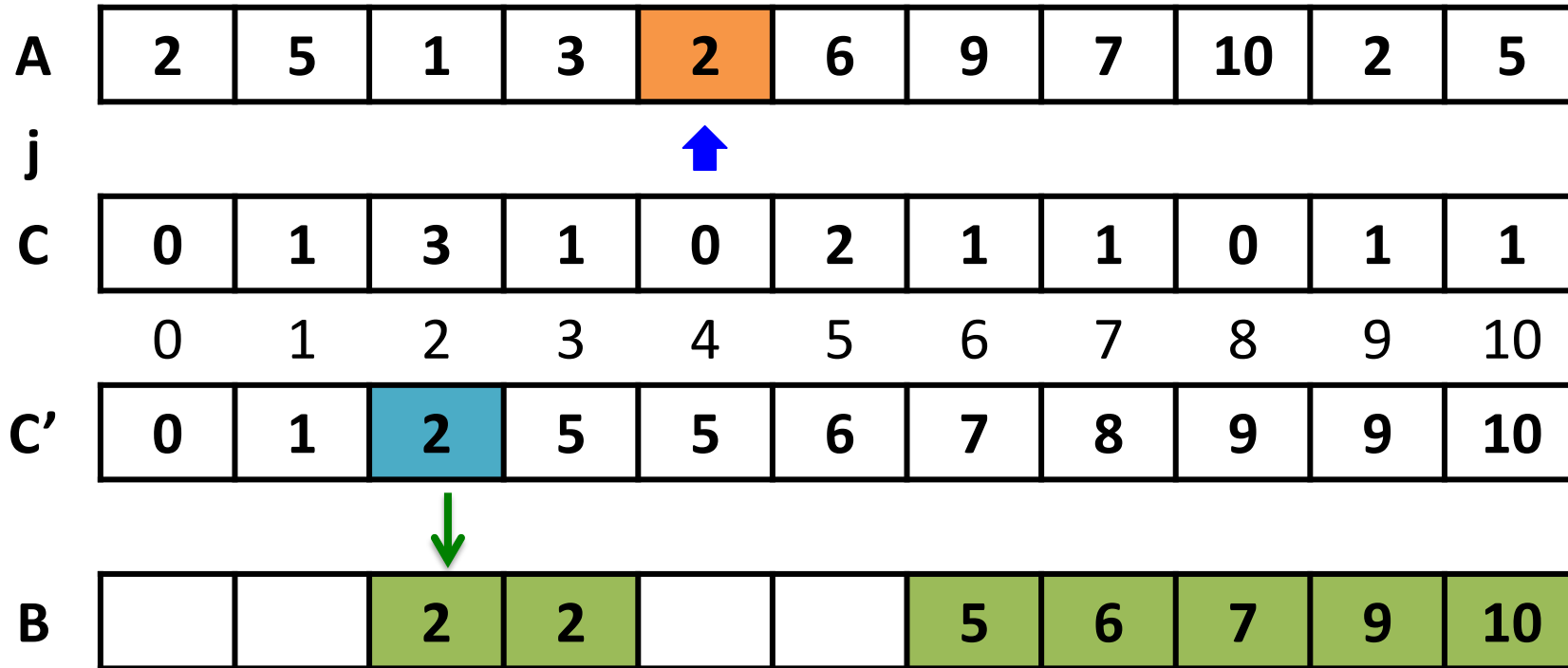
Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

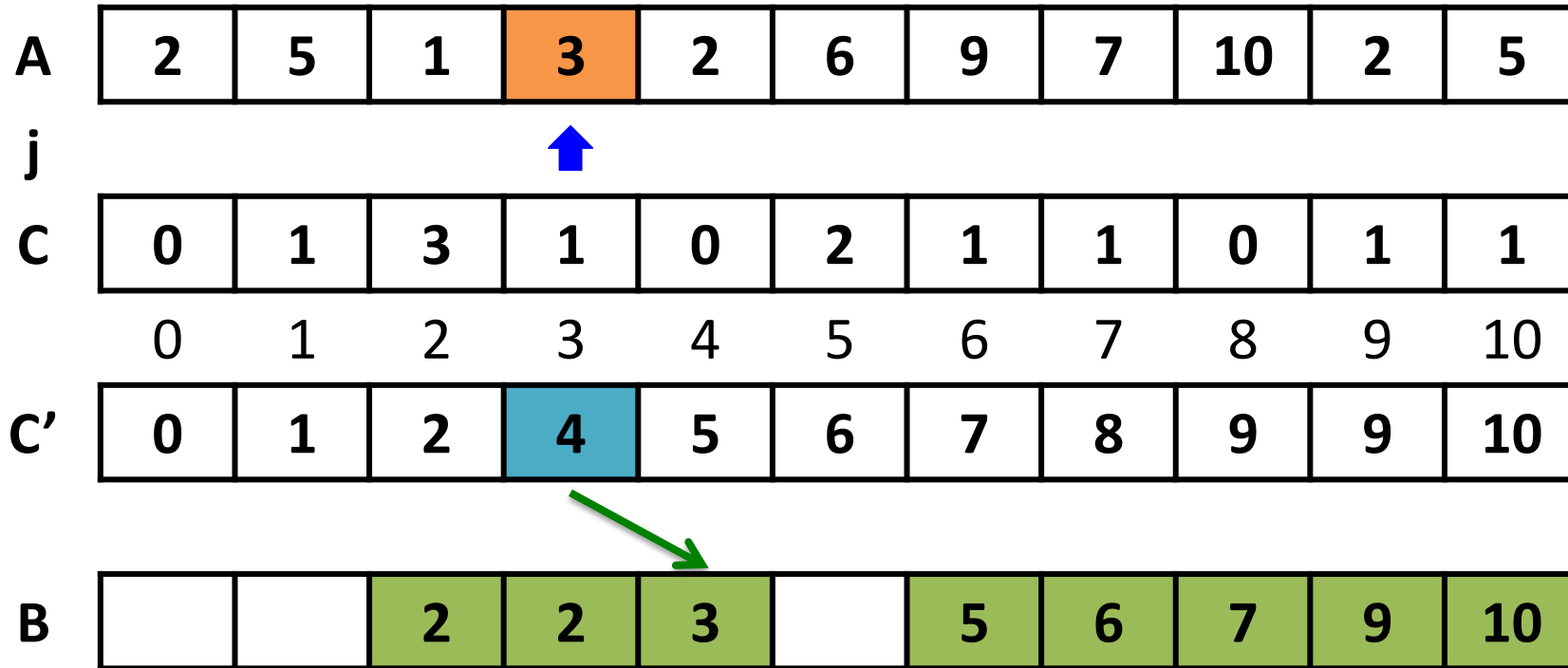
Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

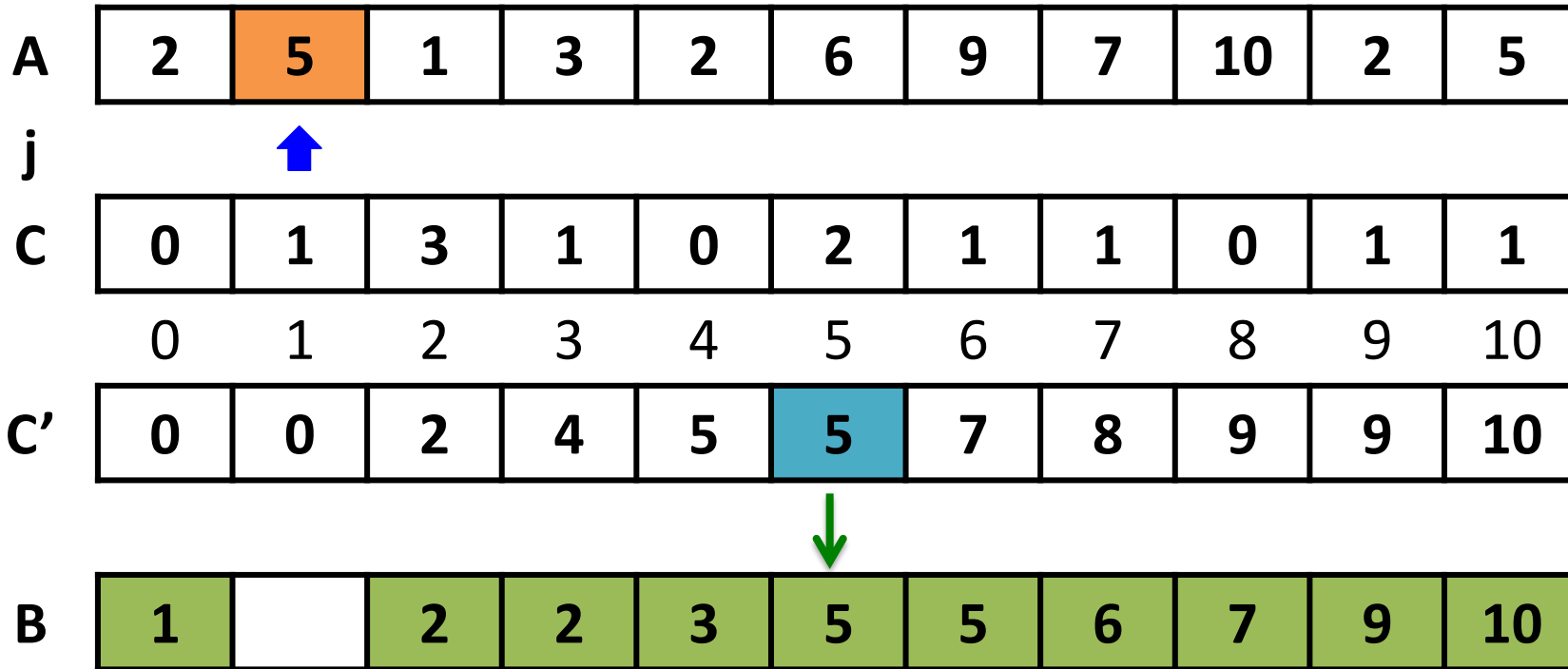
Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j			↑								
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	0	2	4	5	6	7	8	9	9	10
B	1		2	2	3		5	6	7	9	10

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j	↑										
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	0	1	4	5	5	7	8	9	9	10
			↙								
B	1	2	2	2	3	5	5	6	7	9	10

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

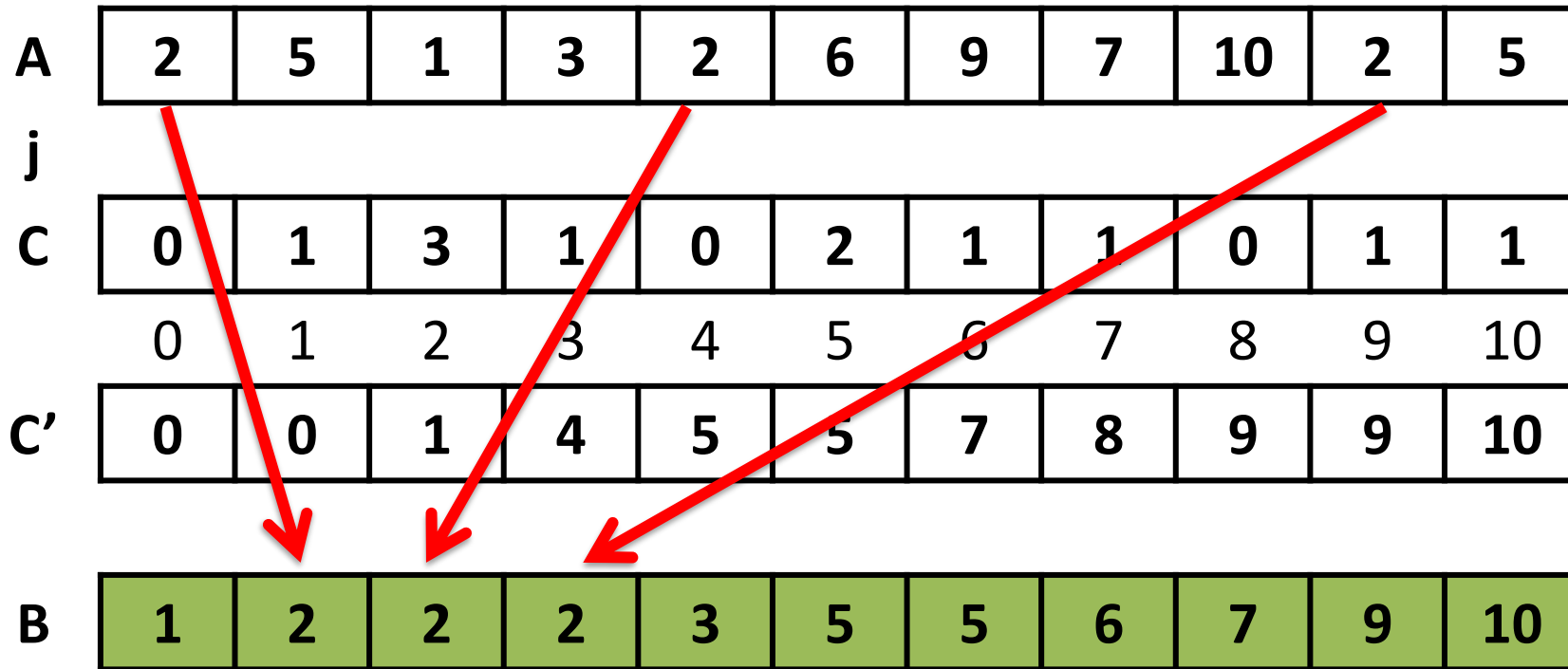
Walk backwards through A to sort it now

A	2	5	1	3	2	6	9	7	10	2	5
j											
C	0	1	3	1	0	2	1	1	0	1	1
	0	1	2	3	4	5	6	7	8	9	10
C'	0	0	1	4	5	5	7	8	9	9	10
B	1	2	2	2	3	5	5	6	7	9	10

$$B[-C'[A[j]]] = A[j]$$

Counting Sort

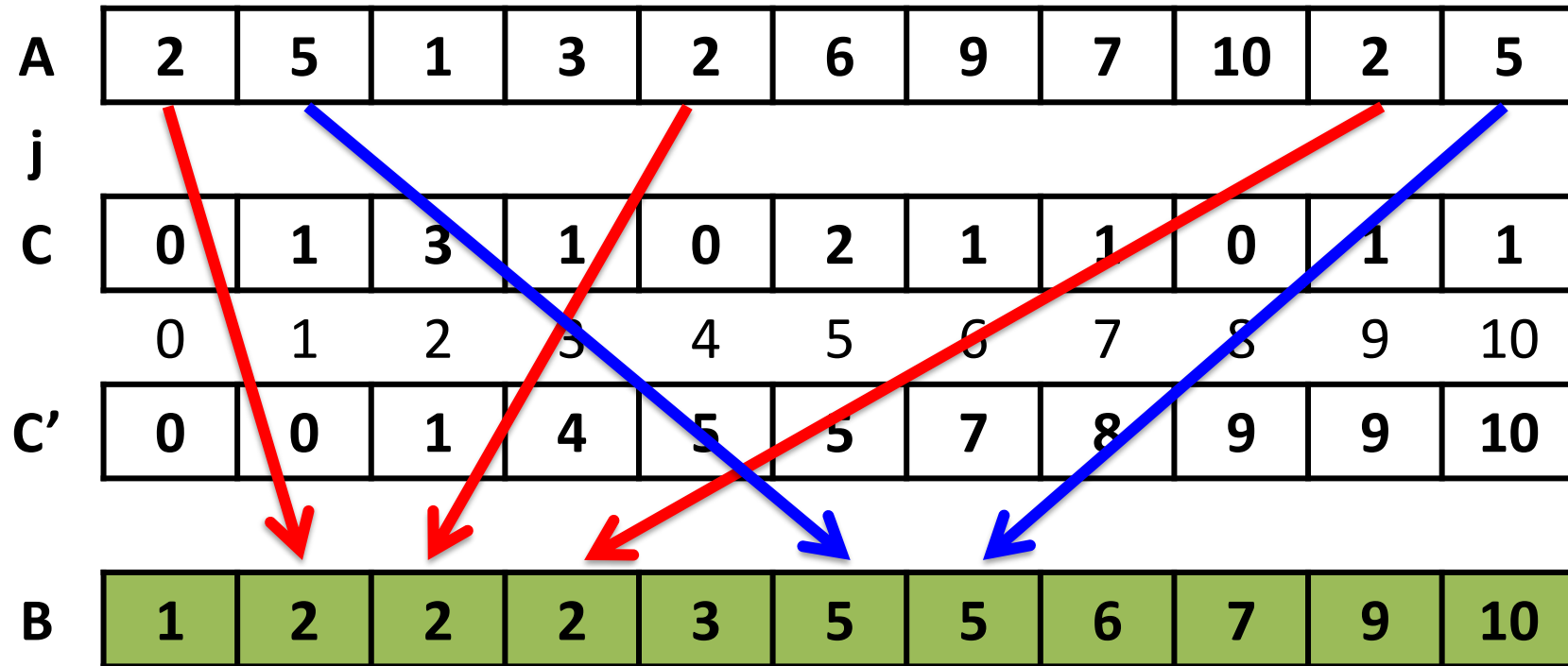
Walk backwards through A to sort it now



$$B[-C'[A[j]]] = A[j]$$

Counting Sort

Walk backwards through A to sort it now



Counting sort is **stable** (but not in-place)

Counting Sort

Theorem 11.7: The **counting sort** algorithm will sort an array with n integers from a range of size k , say $\{a, a + 1, a + 2, \dots, a + k - 1\}$, in time $O(n + k)$

Counting Sort

Theorem 11.7: The **counting sort** algorithm will sort an array with n integers from a range of size k , say $\{a, a + 1, a + 2, \dots, a + k - 1\}$, in time $O(n + k)$

Corollary: We can sort n integers in linear time if the elements come from a range of size $O(n)$.

Radix Sort

Theorem 11.8: For any integer $d > 0$, The **radix sort** can sort an array of n w -bit integers in time

$$O\left(\left(\frac{w}{d}\right)(n + 2^d)\right)$$

Idea: Use counting sort to sort the data by their d least significant bits, then sort the data by their next d bits, then on the next d bits...

(works since counting sort is stable!)

Radix Sort

The **radix sort**

0010	1001	1100
1101	1010	1100
0101	1100	0001
1100	1010	0010
1001	1100	1100
<u>1100</u>	<u>1100</u>	<u>0010</u>
1101	1001	0001
1010	1001	0001

Radix Sort

The **radix sort**

0010	1001	1100
1101	1010	1100
0101	1100	0001
1100	1010	0010
1001	1100	1100
<u>1100</u>	<u>1100</u>	<u>0010</u>
1101	1001	0001
1010	1001	0001

Radix Sort

The **radix sort**

0101	1100	0001
1101	1001	0001
1010	1001	0001
1100	1010	0010
<u>1100</u>	<u>1100</u>	<u>0010</u>
0010	1001	1100
1101	1010	1100
1001	1100	1100

Radix Sort

The **radix sort**

0101	1100	0001
1101	1001	0001
1010	1001	0001
1100	1010	0010
<u>1100</u>	<u>1100</u>	<u>0010</u>
0010	1001	1100
1101	1010	1100
1001	1100	1100

Radix Sort

The **radix sort**

1101	1001	0001
1010	1001	0001
0010	1001	1100
1100	1010	0010
1101	1010	1100
0101	1100	0001
<u>1100</u>	<u>1100</u>	<u>0010</u>
1001	1100	1100

Radix Sort

The **radix sort**

1101	1001	0001
1010	1001	0001
0010	1001	1100
1100	1010	0010
1101	1010	1100
0101	1100	0001
<u>1100</u>	<u>1100</u>	<u>0010</u>
1001	1100	1100

Radix Sort

The **radix sort**

0010	1001	1100
0101	1100	0001
1001	1100	1100
1010	1001	0001
1100	1010	0010
<u>1100</u>	<u>1100</u>	<u>0010</u>
1101	1001	0001
1101	1010	1100

Radix Sort

The **radix sort**

0010	1001	1100
0101	1100	0001
1001	1100	1100
1010	1001	0001
1100	1010	0010
<u>1100</u>	<u>1100</u>	<u>0010</u>
1101	1001	0001
1101	1010	1100

Radix Sort

Theorem 11.8: For any integer $d > 0$, The **radix sort** can sort an array of n w -bit integers in time

$$O\left(\left(\frac{w}{d}\right)(n + 2^d)\right)$$